

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Exercise Program

Date: 10/07/2026

Week No.: 1

Program S. No. 2

Title	Student Marks Data Manipulation and Analysis using Pandas																						
Problem Statement	Basic Student Marks Data Manipulation with Pandas: Create a DataFrame with the columns StudentID , Name , and Marks for five students. Display the original dataset, select only the Name and Marks columns, filter students whose Marks are greater than 70 , add a Grade column based on the marks obtained, sort the DataFrame by Marks in ascending order , and finally calculate the average marks of all students.																						
Objectives	• To create a Pandas DataFrame from a Python dictionary containing student records. • To select specific columns from a DataFrame for focused analysis. • To filter student records using a conditional expression. • To create a Grade column using conditional logic and list comprehension. • To sort student records according to marks. • To calculate the average marks using a Pandas aggregate function.																						
Dataset URL Link / File	<table><tr><th>StudentID</th><th>Name</th><th>Marks</th></tr><tr><td>10</td><td>Hasnain</td><td>45</td></tr><tr><td>11</td><td>John</td><td>86</td></tr><tr><td>12</td><td>Alice</td><td>90</td></tr><tr><td>13</td><td>Sahil</td><td>67</td></tr><tr><td>14</td><td>Cook</td><td>97</td></tr></table>					StudentID	Name	Marks	10	Hasnain	45	11	John	86	12	Alice	90	13	Sahil	67	14	Cook	97
StudentID	Name	Marks																					
10	Hasnain	45																					
11	John	86																					
12	Alice	90																					
13	Sahil	67																					
14	Cook	97																					
Functions Details	Sl. No.	Function	Package	Description of function																			
	1	DataFrame()	pandas	Creates a DataFrame from dictionary data																			
	2	[] / Boolean Indexing	pandas	Selects columns and filters rows																			
	3	sort_values()	pandas	Sorts rows according to Marks																			
	4	mean()	pandas	Calculates the average Marks value																			
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library. Step 2 Create a Python dictionary containing StudentID, Name, and Marks for five students. Step 3 Convert the dictionary into a Pandas DataFrame and display the original dataset. Step 4 Select and display only the Name and Marks columns. Step 5 Filter and display students whose Marks are greater than 70. Step 6 Add a Grade column by assigning grades A to E according to the marks obtained. Step 7 Display the updated DataFrame containing the Grade column. Step 8 Sort the DataFrame by Marks in ascending order and display the result.																						

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Exercise Program

Date: 10/07/2026

Week No.: 1

Program S. No. 2

	Step 9 Calculate and display the average marks using the mean() function.
Program	<pre># Import the Pandas library import pandas as pd # Create a dictionary containing student details dictionary_1 = { "StudentID": [10, 11, 12, 13, 14], "Name": ["Hasnain", "John", "Alice", "Sahil", "Cook"], "Marks": [45, 86, 90, 67, 97] } # Convert the dictionary into a Pandas DataFrame dataFrame = pd.DataFrame(dictionary_1) # Display the original dataset print("Displaying the Original Dataset\n") print(dataFrame, "\n") # Select and display only the Name and Marks columns only_name_column = pd.DataFrame(dataFrame, columns=["Name", "Marks"]) print("Displaying only name and marks columns\n") print(only_name_column, "\n") # Filter students whose marks are greater than 70 marks_greater_than_70 = pd.DataFrame(dataFrame[dataFrame["Marks"] > 70]) print("Displaying students with marks greater than 70\n") print(marks_greater_than_70, "\n") # Assign grades based on the marks obtained dataFrame["Grade"] = ["A" if marks >= 90 else "B" if marks >= 80 else "C" if marks >= 70 else "D" if marks >= 60 else "E" for marks in dataFrame["Marks"]] # Display the updated dataset with the Grade column print("Displaying the updated dataset\n") print(dataFrame, "\n") # Sort the dataset by Marks in ascending order sort_by_marks = dataFrame.sort_values(by="Marks", ascending=True) print("Displaying the sorted dataset\n") print(sort_by_marks, "\n") # Calculate and display the average marks average = dataFrame["Marks"].mean() print("Average Marks \n") print(average)</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Exercise Program

Date: 10/07/2026

Week No.: 1

Program S. No. 2

**Output
Snapshots
With
Explanation/
Observations**

Displaying the Original Dataset

```
***
  StudentID   Name  Marks
0         10  Hasnain    45
1         11    John    86
2         12    Alice    90
3         13    Sahil    67
4         14     Cook    97
```

Figure 2.1 Original Student Dataset

2.2 shows only the Name and Marks columns selected from the original DataFrame. This demonstrates column selection and allows the analysis to focus on the student names and their corresponding marks without displaying StudentID.

Displaying only name and marks columns

```

   Name  Marks
0  Hasnain    45
1    John    86
2    Alice    90
3    Sahil    67
4     Cook    97
```

Figure 2.2 Selected Name and Marks Columns

Displaying students with marks greater than 70

```

  StudentID   Name  Marks
1         11    John    86
2         12    Alice    90
4         14     Cook    97
```

Figure 2.3 Filtering student with marks > 70

Figure 2.3 presents the filtered records for students whose marks are greater than 70. The Boolean condition `dataFrame["Marks"] > 70` retains John, Alice, and Cook because their marks satisfy the specified condition.

2.4 displays the updated DataFrame after adding the Grade column. Grades are assigned using conditional logic: A for marks of 90 or above, B for 80-89, C for 70-79, D for 60-69, and marks below 60.

Displaying the updated dataset

```

  StudentID   Name  Marks Grade
0         10  Hasnain    45     E
1         11    John    86     B
2         12    Alice    90     A
3         13    Sahil    67     D
4         14     Cook    97     A
```

Figure 2.4 Dataset after Adding the Grade Column

Displaying the sorted dataset

```

  StudentID   Name  Marks Grade
0         10  Hasnain    45     E
3         13    Sahil    67     D
1         11    John    86     B
2         12    Alice    90     A
4         14     Cook    97     A
```

2.5 Dataset Sorted by Marks in Ascending Order

Figure 2.5 shows the records sorted from the lowest to the highest marks using `sort_values()`. Hasnain appears first with 45 marks, while Cook appears last with 97 marks. The original row indexes are retained by Pandas.

2.6 displays the average marks calculated using the `mean()` function. The total marks of all students divided by five gives an average of 77.0.

Average Marks

77.0

Figure 2.6 Average Marks

Conclusions

This exercise successfully demonstrated basic student-data manipulation and analysis using Pandas. A DataFrame containing StudentID, Name, and Marks was created from a Python dictionary, and multiple operations were performed on the dataset.

The results showed that:

- The Name and Marks columns were selected successfully for focused analysis.
- Students with marks greater than 70 were identified using Boolean filtering.
- A Grade column was generated using conditional logic based on each student's marks.
- The records were arranged in ascending order of marks using `sort_values()`.
- The average marks of the five students were calculated as 77.0 using `mean()`.

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Exercise Program

Date: 10/07/2026

Week No.: 1

Program S. No. 2

	Overall, the exercise provided practical experience in DataFrame creation, column selection, row filtering, conditional transformation, sorting, and statistical aggregation. These operations are fundamental for data preprocessing and exploratory data analysis in Data Science and Machine Learning applications.
References	1. Wes McKinney, Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter, 3rd Edition, O'Reilly Media, 2022. 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab Link) https://colab.research.google.com/drive/1qBzvtH6ZEis8Jv20WDiYXKI4PXGSXcWJ?usp=sharing 4. Pandas DataFrame API Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html 5. Class lecture link: https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1Ut4qVMUy/view

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore